SatsPath Engine — Walkthrough

What SatsPath Is

SatsPath is a robust backend engine, protocol daemon (`satspathd`), and CLI designed to act as a universal signed payment resolver and router. It is intended to be embedded into existing wallets (via WASM or FFI) or run as a standalone service, acting as the "brain" for resolving identity profiles and optimizing payment routing.

It can: - Resolve a local or peer-registered signed profile via multiple resolution methods (Local Registry, BIP-353 DNS, HTTP Well-Known, Nostr). - Select an optimal payment rail (Lightning, On-chain, Ark) based on live mempool fees and routing rules. - Authenticate and verify hybrid Post-Quantum signatures (ML-DSA-65 + Schnorr). - Fetch real LNURL invoices and BOLT12 offers. - Evaluate Silent Payments (BIP-352) and build BIP-21 on-chain URIs. - Preview swap directives (testnet only).

It cannot (and intentionally does not): - Move funds automatically. - Sign Bitcoin transactions (no PSBT signing). - Broadcast anything to the network. - Store or generate seed phrases. - Execute mainnet swaps.

Architecture

The project has been pruned into a minimal, standalone backend ecosystem focused purely on Rust (`crates/`), Cloudflare workers (`proxy-workers/`), and integration SDKs.

- `satspath-core`: Core models, profile definition, identity keys (ECDSA/Schnorr/PQC), local registry, and resolvers (BIP-353, Nostr, HTTP).
- `satspath-router`: The routing engine that queries live fees (with redundant oracles like mempool.space and mempool.ninja) and selects the best payment path.
- `satspath-pqc`: Hybrid cryptographic suite combining classical signatures with ML-DSA-65.
- `satspathd`: The standalone SatsPath daemon. It features zero-configuration authentication (auto-generating an `admin.macaroon` token) and a secure API middleware.
- `satspath-wasm`: WASM bindings that allow embedding the SatsPath resolver and router into frontend applications.
- `satspath-cli`: Command-line interface for human-readable interactions (ASCII QR codes, profile management, JSON quoting).
- `satspath-swaps`: Experimental scaffold for Boltz Exchange v2 swap integration (testnet intent preview only).

Security and Cryptography

SatsPath is built with a strict Zero-Trust model:

- **Post-Quantum Cryptography (PQC):** Uses a hybrid signature scheme (`ML-DSA-65-Schnorr`) for generating and verifying identity keys. The engine validates that keys and signatures conform to these robust standards, while keeping execution times in the microsecond range.
- **SSRF Protection:** Resolvers strictly validate URLs and block loopback, private, and internal metadata IP ranges (e.g., `169.254.169.254`) to prevent malicious profile endpoints from exploiting internal networks.
- **Nostr Concurrency & Tombstoning:** Downloads profiles from multiple Nostr relays concurrently to ensure the most recent sequence is used, effectively preventing downgrade attacks. It strictly rejects revoked (tombstoned) profiles.
- **Safe Persistence:** All local state (`.satspath/`) uses SHA-256 keyed storage. Sensitive swap material is stored using AES-256-GCM encryption, and plaintext writing is strictly guarded against.

Supported Payment Rails

1. **Lightning Network:** Selected for smaller amounts ($<$ 100k sats). It handles LNURL-pay two-step fetches and parses BOLT11 invoices to verify amounts.

2. **On-chain:** Selected for larger amounts when fees are acceptable. Includes support for Silent Payments (`sp1...` keys) which are seamlessly integrated into the generated `bitcoin:` URIs.
3. **Ark:** Fallback for when fees are high. Provides Ark payment pointers. (Client-side DAG validation is delegated to the integrating wallet).
4. **BOLT12:** An HTTP proxy scaffold (`proxy-workers/bolt12`) is available to resolve BOLT12 offers to real BOLT11 invoices asynchronously when native LNURL is unavailable.

What is Implemented vs. What is Not

Feature	Status
Signed profile resolution (Nostr, HTTP, Local, BIP-353)	
Hybrid Identity Signature Verification (PQC ML-DSA + Schnorr)	
SSRF-protected Resolvers	
Live mempool fee fetch (mempool.space / mempool.ninja)	
Lightning rail selection (amount $<$ 100k sats)	
On-chain rail (fastestFee 20 sat/vB)	
Ark fallback (high fees)	
LNURL-pay two-step invoice fetch	
BOLT12 HTTP proxy resolution	
Silent Payments (BIP-352) URI injection	
Terminal QR code (Dense1x2 unicode)	
LocalPeerRegistry (SHA-256 keyed, no raw email)	
SwapStore AES-256-GCM encryption & sensitive guards	
Boltz API client & Swap creation (testnet scaffolding)	scaffold
Claim/Refund transaction construction	Out of scope
PSBT signing	Out of scope
Ark VTXO DAG validation	Delegated to Wallet
Mainnet swap execution	Intentionally out of scope

Getting Started (Dockerized Environment)

A `docker-compose.yml` file is provided to quickly launch and auto-configure the `satspathd` daemon via a lightweight Multi-Stage Build, making it trivial to deploy a trusted local registry node.

```
make build
make up
make logs
```